

AI 기반 감정 분석 음악 추천 시스템

시스템 설계 문서 v1

작성일	2026-05-13
버전	v1.0
작성자	CWNU CE 박우현, 신성민, 정원준
강의명	소프트웨어공학 (2026년 1학기)
단계	W12 1단계 제출 — 설계 통합 문서

개정 이력

버전	일자	작성자	변경 사항
v1.0	2026-05-13	박우현, 신성민, 정원준	최초 작성 — 아키텍처 4+1 뷰 초안, 클래스·시퀀스 다이어그램 신규, 컴포넌트 사전, 데이터 모델, API 초안, ADR 색인, NFR 전략 포함

목차

- 아키텍처 개요 (4+1 뷰 초안)
- 시스템 컨텍스트
- 컴포넌트 사전 (도메인 어휘)
- 클래스 다이어그램
- 시퀀스 다이어그램 (UC-03)
- 기술 스택
- 데이터 모델
- API 엔드포인트 초안
- 의사결정 색인 (ADR)
- 비기능 요구사항 충족 전략

1. 아키텍처 개요 (4+1 뷰 초안)

1.1 Logical View (논리 뷰)

시스템은 크게 **클라이언트**, **백엔드**, **ML 서버**, **외부 서비스**, **데이터베이스** 다섯 계층으로 구성된다. 클라이언트의 **VoiceCapture** 는 음성을 수집하여 백엔드로 전송하고, **RecommendationVisualizer** 는 추천 결과를 2D 차트와 리스트로 렌더링한다. 백엔드는 **STTService** , **ContextAnalyzer** , **EmotionFusion** , **RecommendationEngine** , **FeedbackLogger** 로 이루어지며, **EmotionClassifier (wav2vec2)**는 별도 ML 서버에서 동작한다. 도메인 엔티티 (User, Music, Recommendation, Feedback)는 PostgreSQL에 영속화된다.

1.2 Process View (프로세스 뷰)

UC-03의 핵심 흐름은 **듀얼 트랙 병렬 처리**다. 음성 수신 후 백엔드는 **asyncio**를 이용해 (a) ML 서버 감정 분류와 (b) STT→LLM 맥락 분석을 동시에 실행한다. 두 결과가 모두 준비되면 **EmotionFusion** 이 통합 쿼리를 생성하고, **RecommendationEngine** 이 카탈로그 유사도 매칭을 수행한다. 전체 응답 목표는 $P95 \leq 3\text{초}$ (NFR1.1)이며, Redis 캐시로 동일 입력 재요청 시 단축된다.

1.3 Development View (개발 뷰)

코드베이스는 **/client** (Electron + Next.js, TypeScript), **/backend** (FastAPI, Python), **/ml** (wav2vec2 학습·서빙, Python), **/infra** (Docker Compose, Blue-Green 구성) 네 모듈로 분리된다. CI/CD는 GitHub Actions에서 **lint(ESLint, Ruff)** + 단위 테스트를 자동 실행하며, PR 머지 시에만 배포 파이프라인이 기동된다. STT는 **STTService** 인터페이스와 **WhisperLocalAdapter** / **WhisperApiAdapter** 두 구현체로 어댑터 패턴을 적용해 교체 용이성을 확보한다.

1.4 Deployment View (배포 뷰)

백엔드와 ML 서버는 자체 서버에서 Docker 컨테이너로 운영된다. Blue-Green 배포 방식을 채택해 새 버전을 Green 컨테이너에 배포하고 nginx가 트래픽을 전환함으로써 다운타임 0초를 보장한다 (NFR2.2). Electron 클라이언트는 **electron-builder** 로 빌드된 배포본을 사용자가 로컬에 설치하며, 서버 접근은 tailscale 메시 VPN을 통해 이루어진다.

1.5 Use Case View (유스케이스 뷰)

핵심 유스케이스는 **UC-03 음성으로 음악 추천 받기**다. 사용자 액터가 음성을 입력하면 시스템의 모든 서브시스템이 협력하여 추천 결과를 반환한다. 관련 UC는 UC-01(회원가입), UC-02(로그인), UC-04(재생), UC-05(피드백), UC-08(카탈로그 관리, 관리자 액터)이며, UC-03이 모든 설계 결정의 기준점이 된다.

아키텍처 다이어그램

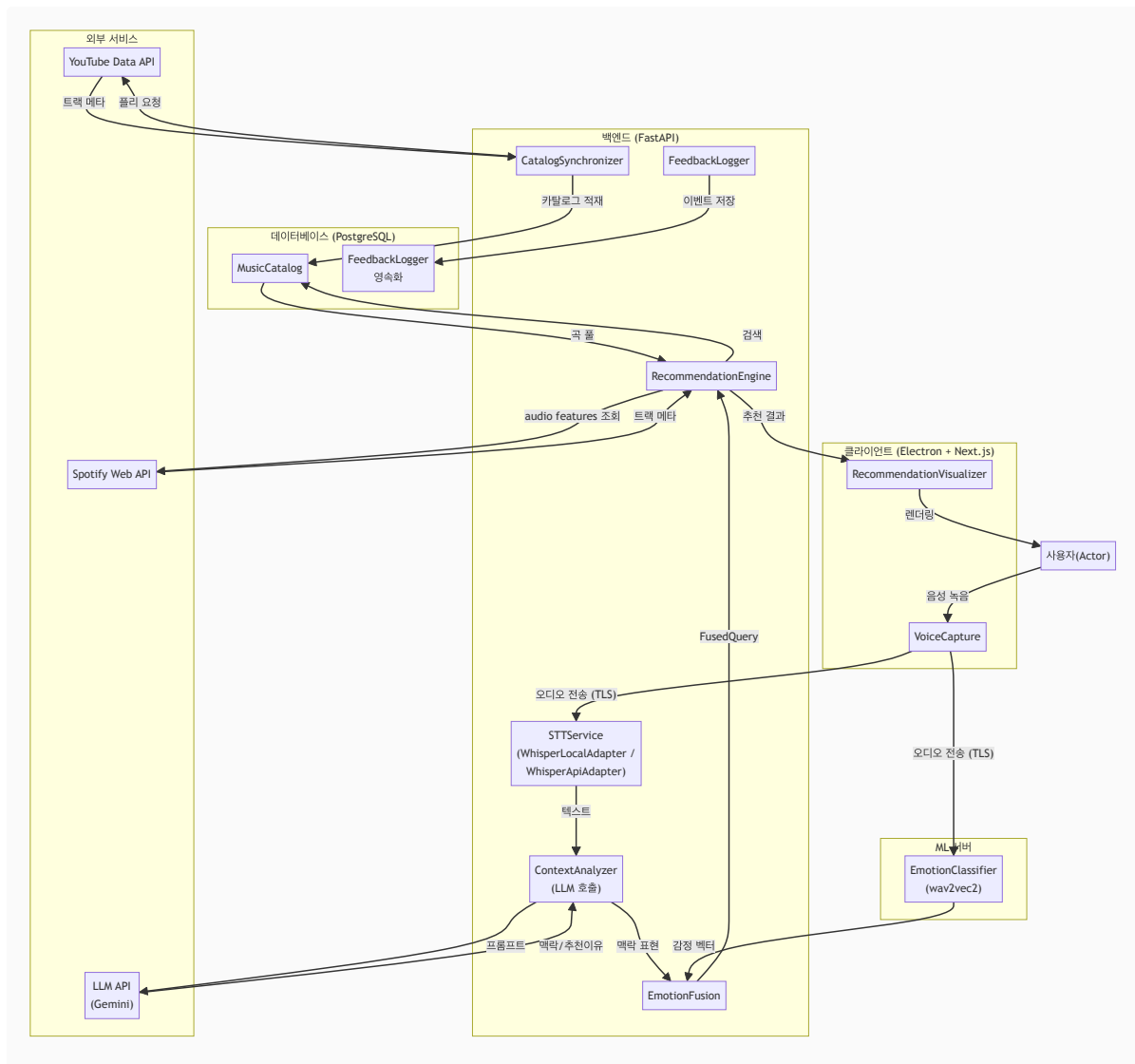


그림 1. 시스템 아키텍처 다이어그램 — 컴포넌트 및 데이터 흐름

2. 시스템 컨텍스트

시스템 컨텍스트 다이어그램 — AI 감정분석 음악추천 시스템

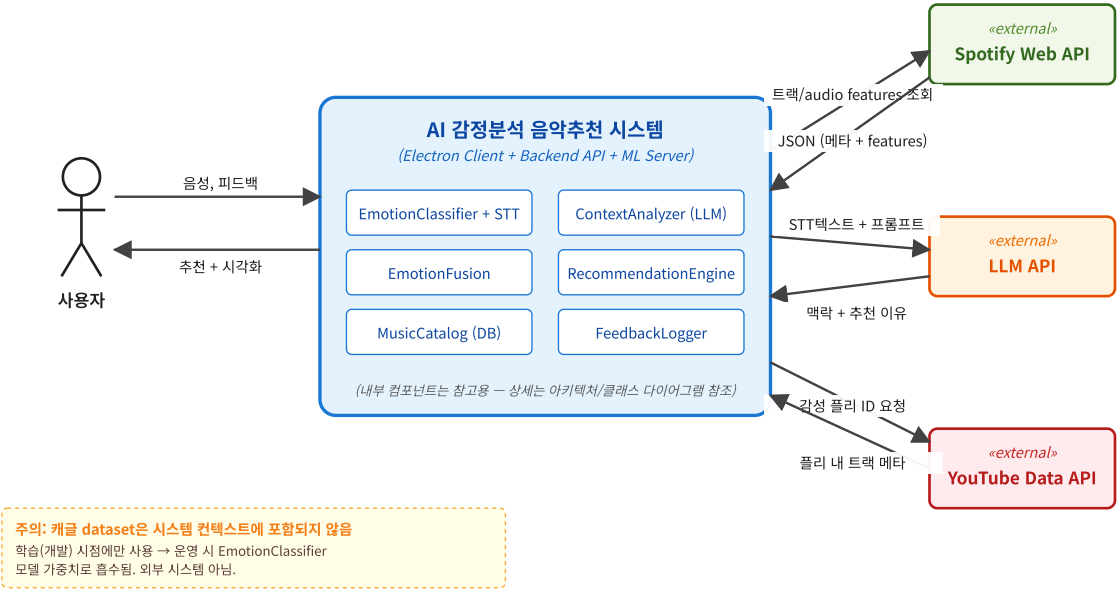


그림 2. 시스템 컨텍스트 다이어그램 (SRS § 3 기반)

2.1 외부 인터페이스 요약

인터페이스	방향	데이터	빈도
사용자 → 시스템	inbound	음성 (WAV/WebM), 피드백 (좋아요/싫어요)	요청당
시스템 → 사용자	outbound	추천 곡 리스트, 2D 차트, LLM 설명 텍스트	요청당
시스템 → Spotify Web API	outbound	트랙 검색, audio features 조회 요청	추천당 1~10회
Spotify Web API → 시스템	inbound	JSON (트랙 메타 + audio features)	위에 대응
시스템 → LLM API	outbound	STT 텍스트 + 분석 프롬프트	추천당 1~2회
LLM API → 시스템	inbound	맥락 표현 + 추천 이유 텍스트	위에 대응
시스템 → YouTube Data API	outbound	감성 플리 ID, 트랙 메타 요청	카탈로그 동기화 시
YouTube Data API → 시스템	inbound	플리 내 트랙 메타	위에 대응

3. 컴포넌트 사전 (도메인 어휘)

후속 다이어그램(클래스, 시퀀스, 아키텍처)에서 일관되게 사용하는 명칭이다. SRS § 7 컴포넌트 사전을 기준으로 한다.

컴포넌트	위치	책임
VoiceCapture	Client (Electron)	MediaRecorder로 음성 녹음 및 TLS 전송
EmotionClassifier	ML Server	캐글 학습 wav2vec2 모델, 음성 → 감정 벡터
STTService	Backend	음성 → 텍스트 변환 (어댑터 패턴: WhisperLocalAdapter / WhisperApiAdapter)
ContextAnalyzer	Backend (LLM API 호출)	텍스트 → 맥락 표현 추출 (Gemini LLM)
EmotionFusion	Backend	감정 벡터 + 맥락 표현 → 통합 쿼리 생성
RecommendationEngine	Backend	Spotify audio features 기반 유사도 매칭, 상위 K=10 선정
MusicCatalog	Backend (DB)	Spotify + YouTube 시드로 구축된 곡 풀, 검색 및 적재
FeedbackLogger	Backend	좋아요/싫어요/재생 이벤트 기록 및 집계
RecommendationVisualizer	Client (Electron)	2D 감정-음악 매핑 차트 + LLM 이유 설명 렌더링
CatalogSynchronizer	Backend (Scheduler)	YouTube 플리 → Spotify 매칭 → 카탈로그 적재 (주기적 실행)

4. 클래스 다이어그램

SRS § 7의 10개 컴포넌트와 핵심 도메인 엔티티를 UML 클래스 다이어그램으로 표현한다. STTService 는 어댑터 패턴으로 설계되어 WhisperLocalAdapter 와 WhisperApiAdapter 두 구현체가 인터페이스를 구현한다 (ADR-0002).

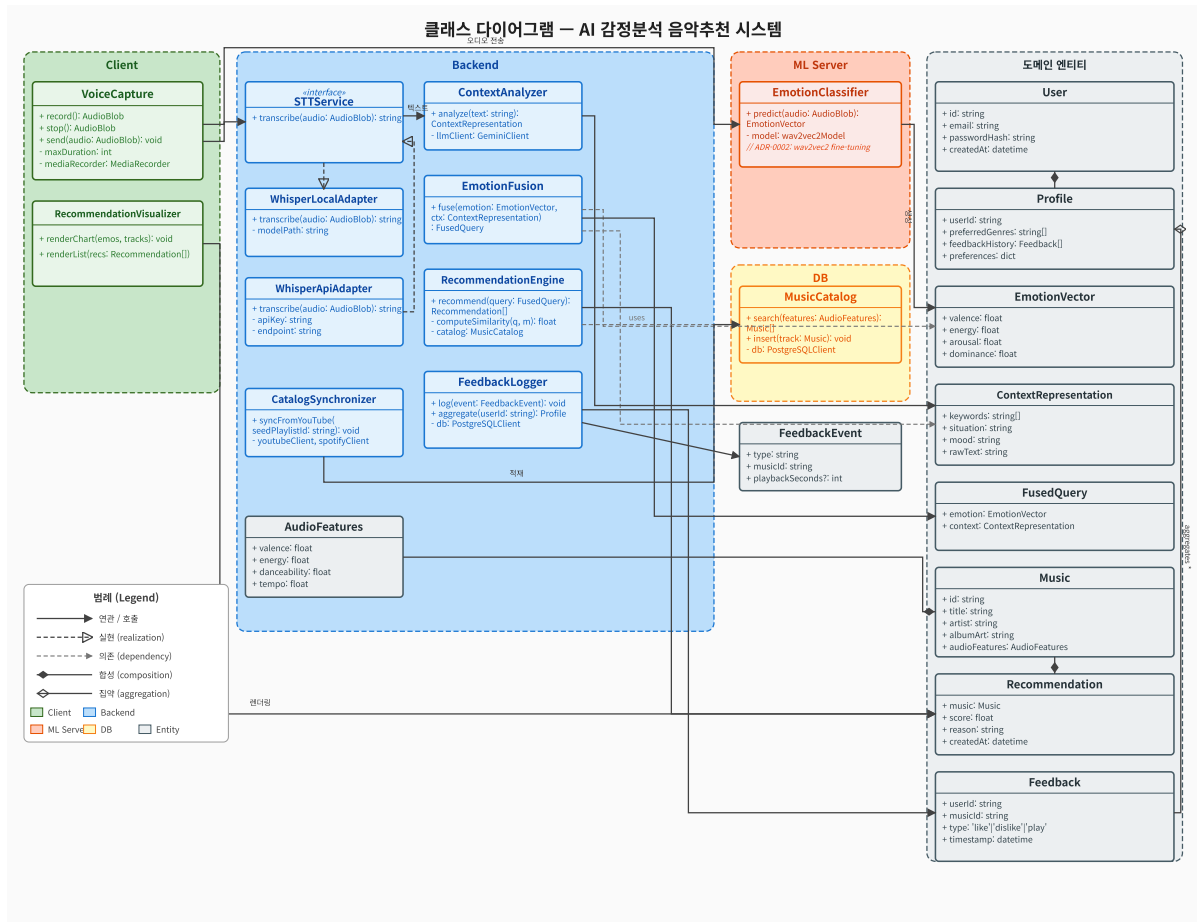


그림 3. 클래스 다이어그램 — 10개 컴포넌트 + 도메인 엔티티 (SVG, docs/회의록/design/diagrams/class.svg)

5. 시퀀스 다이어그램 (UC-03 음성으로 음악 추천 받기)

SRS § 6.2 UC-03의 주 시나리오 11단계를 시퀀스 다이어그램으로 표현한다. 6단계의 듀얼 트랙 병렬 처리는 **par** 프레임으로 나타내고, LLM·ML 장애 대체 시나리오는 **alt** 프레임으로 표현한다.

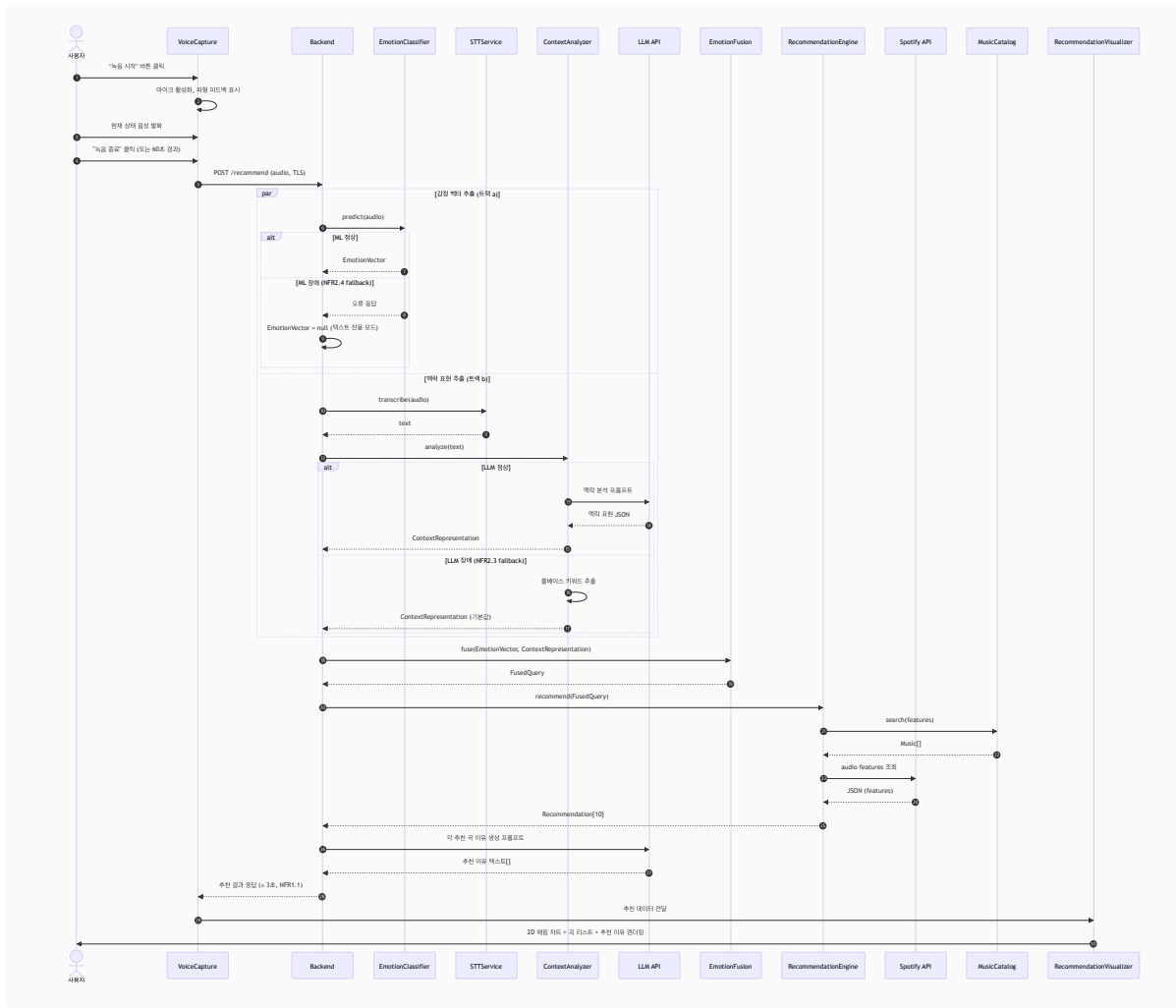


그림 4. UC-03 시퀀스 다이어그램 — par 듀얼 트랙 + alt fallback 분기 (신규 작성)

6. 기술 스택

ADR-0002에서 확정된 8개 영역의 기술 결정과 근거를 정리한다.

영역	채택 기술	대안 (탈락)	근거 (한 줄)
LLM	gemini-3.1-flash-lite-preview	GPT-4o, Claude Haiku, 로컬 Llama	저비용 + 빠른 응답 + 한국어 가용; 모델 ID는 환경변수로 분리해 preview → GA 전환 대응
STT	어댑터 패턴 + Whisper Small (로컬) / Whisper API	Clova, 단일 Whisper 백엔드	STTService 인터페이스로 캡슐화, 운영 중 비용·지연 trade-off에 따라 교체 가능 (NFR2.3 fallback 동일 패턴)
음성 감정 ML	wav2vec2 (fine-tuning)	CNN+멜스펙트로그램, LSTM	사전학습 모델 품질 우수; 캐글 dataset fine-tuning 워크플로 친숙; 목표 정확도 $\geq 70\%$ (NFR4.3)
백엔드	FastAPI (Python)	Express, Flask	비동기 IO + Pydantic + ML과 동일 Python 런타임 → 듀얼 트랙 asyncio 병렬 처리에 자연스러움
데이터베이스	PostgreSQL	MongoDB	관계형 안정성 + JSONB 로 피드백·추천 이력 반정형 데이터 흡수
프론트엔드	Electron + Next.js	React 단독, Vue	ADR-0001 Electron 위에 Next.js 라우팅·컴포넌트 생태계 확정; 팀 JS/TS 역량 직접 활용
배포	자체 서버 (Blue-Green)	Heroku/Vercel, AWS/GCP	API/LLM 비용 통제 + 무중단 배포 직접 구성 (NFR2.2) + 시연 외부 종속 없음
네트워크	tailscale (메시 VPN)	nginx + Let's Encrypt, Cloudflare Tunnel	자체 서버 ↔ 개발자/CI 메시 VPN; SSH·Public IP 노출 최소화; 비상용 SSH 키 1개 별도 보존

7. 데이터 모델

핵심 엔티티와 관계를 ER 다이어그램으로 표현한다. PK는 PK, FK는 FK로 표시한다.

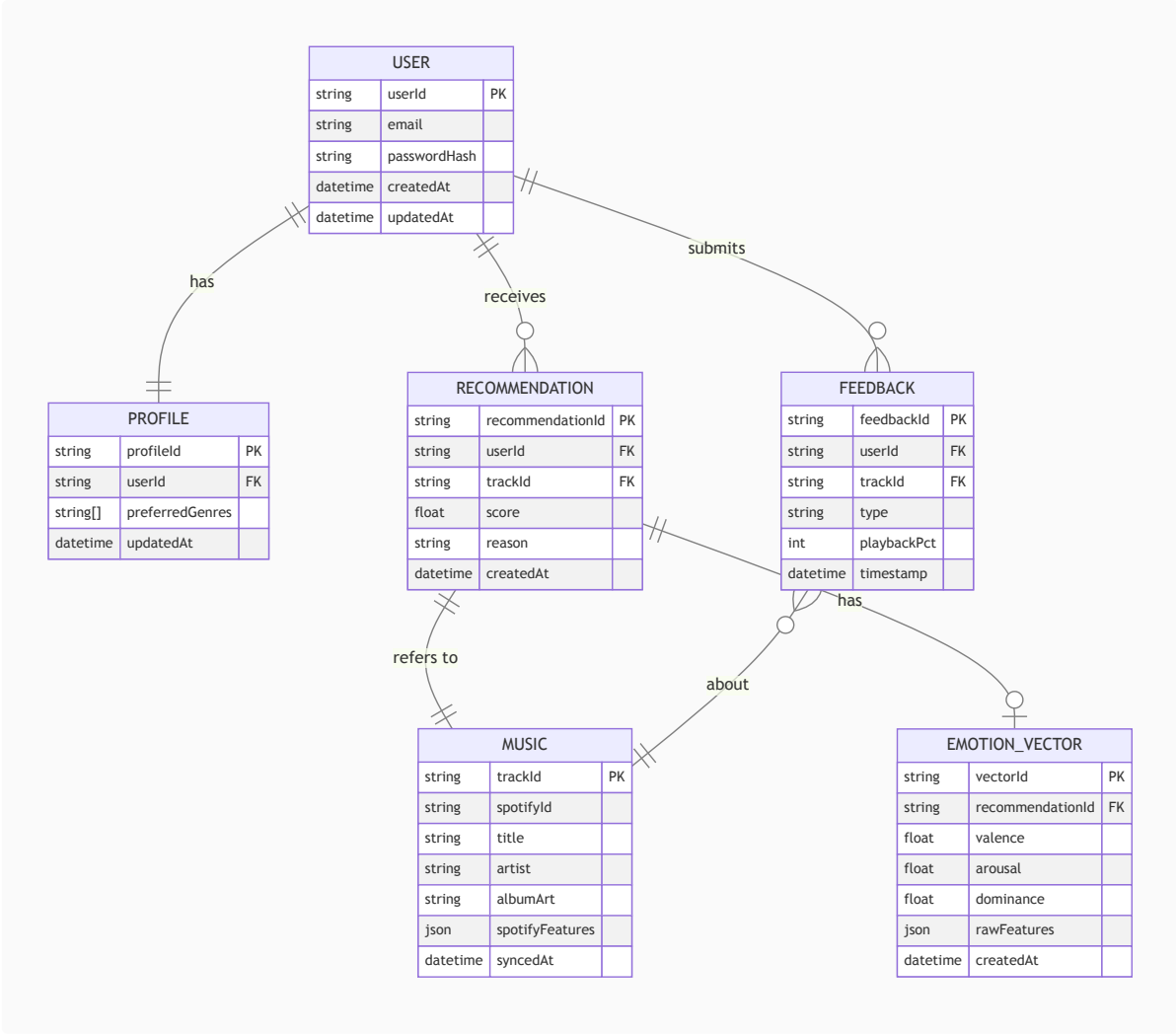


그림 5. 데이터 모델 ER 다이어그램 — 핵심 6개 엔티티

엔티티 설명

엔티티	설명	주요 컬럼
USER	시스템 사용자 계정	userId (PK), email, passwordHash (bcrypt)
PROFILE	사용자 선호 및 누적 특성	userId (FK), preferredGenres (배열)
EMOTION_VECTOR	분석된 감정 벡터 (추천당 1개, 원본 음성 미저장)	valence, arousal, dominance, rawFeatures (JSONB)
MUSIC	카탈로그 곡 메타데이터	spotifyId, spotifyFeatures (JSONB)
RECOMMENDATION	추천 이력 (사용자별 조회 가능)	userId (FK), trackId (FK), score, reason
FEEDBACK	좋아요/싫어요/재생 이벤트	type (like/dislike/playback), playbackPct

8. API 엔드포인트 초안

Method	Path	입력	출력	관련 컴포넌트
POST	/auth/signup	JSON: email, password	201 Created: userId, token	FR1.1
POST	/auth/login	JSON: email, password	200 OK: JWT token	FR1.2
POST	/auth/logout	Header: Authorization (Bearer)	204 No Content	FR1.2
POST	/recommend	multipart/form-data: audio (WAV/WebM, ≤ 60s)	200 OK: recommendations[] (track, score, reason, emotionVector)	VoiceCapture, EmotionClassifier, STTService, ContextAnalyzer, EmotionFusion, RecommendationEngine (FR2~FR5)
POST	/feedback/like	JSON: trackId, recommendationId	201 Created	FeedbackLogger (FR6.1)
POST	/feedback/dislike	JSON: trackId, recommendationId	201 Created	FeedbackLogger (FR6.1)
POST	/feedback/playback	JSON: trackId, event (start/end/complete), playbackPct	201 Created	FeedbackLogger (FR6.2)
GET	/history	Query: n=10 (최근 N 개)	200 OK: recommendations[] with feedback	FeedbackLogger, RecommendationEngine (FR6.5)
POST	/admin/catalog/sync	JSON: playlistId (YouTube)	202 Accepted: jobId	CatalogSynchronizer, MusicCatalog (FR7)
GET	/health	없음	200 OK: {status, version, uptime}	전체 시스템 상태 확인 (NFR2.1)

인증: /auth/signup, /auth/login, /health 를 제외한 모든 엔드포인트는 Authorization: Bearer <JWT> 헤더 필수.

보안: 모든 통신은 TLS 1.2+ 적용 (NFR3.1). 음성 원본은 분석 완료 후 서버에서 즉시 폐기 (NFR3.2).

9. 의사결정 색인 (ADR)

ADR	상태	요약	파일
ADR-0001	Accepted	크로스플랫폼 마이크 접근 및 팀 JS/TS 역량을 고려해 클라이언트 플랫폼으로 Electron을 채택; PWA·네이티브·Tauri·Flutter 대안 탈락	docs/회의록/decisions/0001-electron-as-client-platform.md
ADR-0002	Accepted	8개 기술 영역(LLM·STT·ML·백엔드·DB·프론트엔드·배포·네트워크) 일괄 확정; 비용 통제·Python 런타임 동질성·Blue-Green 배포 직접 구성이 핵심 동인	docs/회의록/decisions/0002-tech-stack.md

10. 비기능 요구사항 충족 전략

NFR	요구 수준	충족 전략
NFR1 성능	응답 P95 ≤ 3초, ML 추론 ≤ 1.5초	• 듀얼 트랙(감정 분류 + STT→맥락 분석) asyncio 병렬 실행으로 직렬 대비 50% 이상 단축 • 동일 입력 추천 결과 Redis 캐싱 (TTL 1시간, FR4.4) • EmotionClassifier는 ML 전용 서버에서 GPU 서빙, 추론 시간 격리
NFR2 가용성	API 가용성 ≥ 99%, 무중단 배포	• Blue-Green 배포: Green 컨테이너에 신 버전 준비 후 nginx 트래픽 전환 → 다운타임 0초 • LLM 장애 시 룰베이스 키워드 추출 fallback (NFR2.3) • ML 장애 시 텍스트 기반 추천만으로 응답 지속 (NFR2.4) • Spotify API 장애 시 캐시된 카탈로그에서 매칭
NFR3 보안	TLS 1.2+, 음성 즉시 폐기, bcrypt	• 모든 API 통신 TLS 1.2+ 강제 (tailscale 메시 + nginx TLS termination) • 음성 원본은 분석(EmotionClassifier + STT) 완료 즉시 서버 메모리에서 폐기; DB에 감정 벡터만 저장 (NFR3.2) • 비밀번호 bcrypt(cost ≥ 12) 해싱 (NFR3.4) • API 키 전부 환경변수 관리, Electron contextIsolation: true + nodeIntegration: false 적용
NFR4 품질	ML 정확도 ≥ 70%, 좋아요율 ≥ 50%, 커버리지 ≥ 70%	• wav2vec2 캐글 테스트셋 정확도 측정을 CI 파이프라인에 통합 • 피드백 누적 → Recommendation Engine 가중치 갱신으로 개인화 향상 (FR6.4) • Jest(클라이언트) + Pytest(백엔드/ML) 커버리지 리포트, PR 머지 조건으로 ≥ 70% 강제
NFR5 사용성	첫 추천 ≤ 3 클릭, 색맹 대응 팔레트	• 메인 화면에 "녹음 시작" 버튼 직접 노출 → 로그인 후 1클릭으로 녹음, 1클릭으로 종료, 자동 추천 → 3 클릭 이내 • 2D 감정-음악 차트에 Okabe-Ito 색맹 대응 팔레트 적용 (NFR5.2)
NFR6 이식성	Windows 10+, macOS 12+, Ubuntu 22+	• Electron 단일 코드베이스 → electron-builder로 3-플랫폼 바이너리 자동 빌드 • CI 매트릭스: Windows / macOS / Ubuntu 병렬 빌드 검증